
Lecture on MapReduce and Spark

Asaf Cidon



Today: MapReduce and Spark

- MapReduce
 - Analytics programming interface
 - Word count example
 - Chaining
 - Reading/write from/to HDFS
 - Dealing with failures
- Spark
 - Motivation
 - Resilient Distributed Datasets (RDDs)
 - Programming interface
 - Transformations and actions

Borrowed from Jeff Ullman, Cristiana Amza and Indranil Gupta

MapReduce



MapReduce and Hadoop

- SQL and ACID are a very useful set of abstractions
- But: unnecessarily heavy for many tasks, hard to scale
- MapReduce is a more limited style of programming designed for:
 1. Easy parallel programming
 2. Invisible management of hardware and software failures
 3. Auto-management of very large-scale data
- It has several implementations, including Hadoop, Flink, and the original Google implementation just called “MapReduce.”
- Not to be too confusing, but it is also used in Spark, which we will talk about later

MapReduce in a Nutshell

- A MapReduce job starts with a collection of input elements of a single type.
 - Technically, all types are key-value pairs.
- Apply a user-written **Map function** to each input element, in parallel.
 - **Mapper** applies the Map function to a single element.
 - Many mappers grouped in a **Map task** (the unit of abstraction for the scheduler)
 - Usually a single Map task is run on a single node/server
- The output of the Map function is a set of 0, 1, or more *key-value pairs*.
- The system sorts all the key-value pairs by key, forming key-(list of values) pairs.

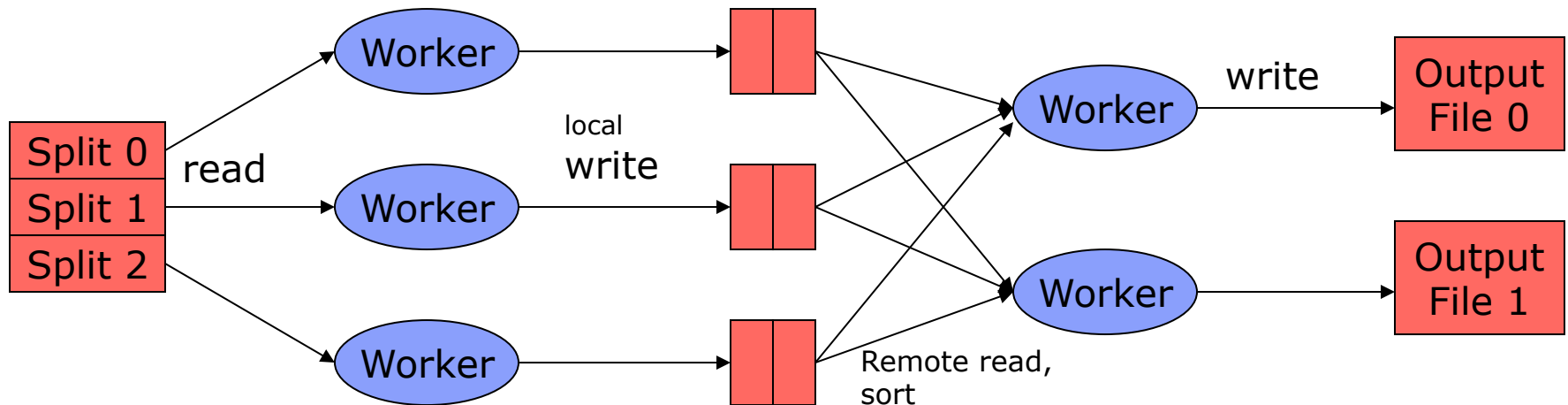
In a Nutshell – (2)

- Another user-written function, the **Reduce function**, is applied to each key-(list of values).
 - Application of the Reduce function to one key and its list of values is a **reducer**.
 - Often, many reducers are grouped into a **Reduce task**.
- Each reducer produces some output, and the output of the entire job is the union of what is produced by each reducer.

MapReduce workflow

Input Data

Output Data



Map

extract something you
care about from each
record

Reduce

aggregate,
summarize, filter,
or transform

Example: Word Count

- We have a large text file, which contains many documents
- The documents contain words separated by whitespace
- Count the number of times each distinct word appears in the file

Word Count Using MapReduce

```
map(key, value):
```

```
// key: document ID; value: text of document
```

```
  FOR (each word w IN value)
```

```
    emit(w, 1);
```

```
reduce(key, value-list):
```

```
// key: a word; value-list: a list of integers
```

```
  result = 0;
```

```
  FOR (each integer v on value-list)
```

```
    result += v;
```

```
  emit(key, result);
```

Expect to be all 1's, but
"combiners" allow local
summing of
integers with the same
key before passing to
reducers.



Mapper

- Reads in input pair `<Key, Value>`
- Outputs a pair `<K', V'>`
 - Let's count number of each word in user queries (or Tweets/Blogs)
 - The input to the mapper will be `<queryID, QueryText>`:
`<Q1, "The teacher went to the store. The store was closed; the store opens in the morning. The store opens at 9am." >`
 - The output would be:
`<The, 1> <teacher, 1> <went, 1> <to, 1> <the, 1> <store, 1> <the, 1> <store, 1> <was, 1> <closed, 1> <the, 1> <store, 1> <opens, 1> <in, 1> <the, 1> <morning, 1> <the 1> <store, 1> <opens, 1> <at, 1> <9am, 1>`

Reducer

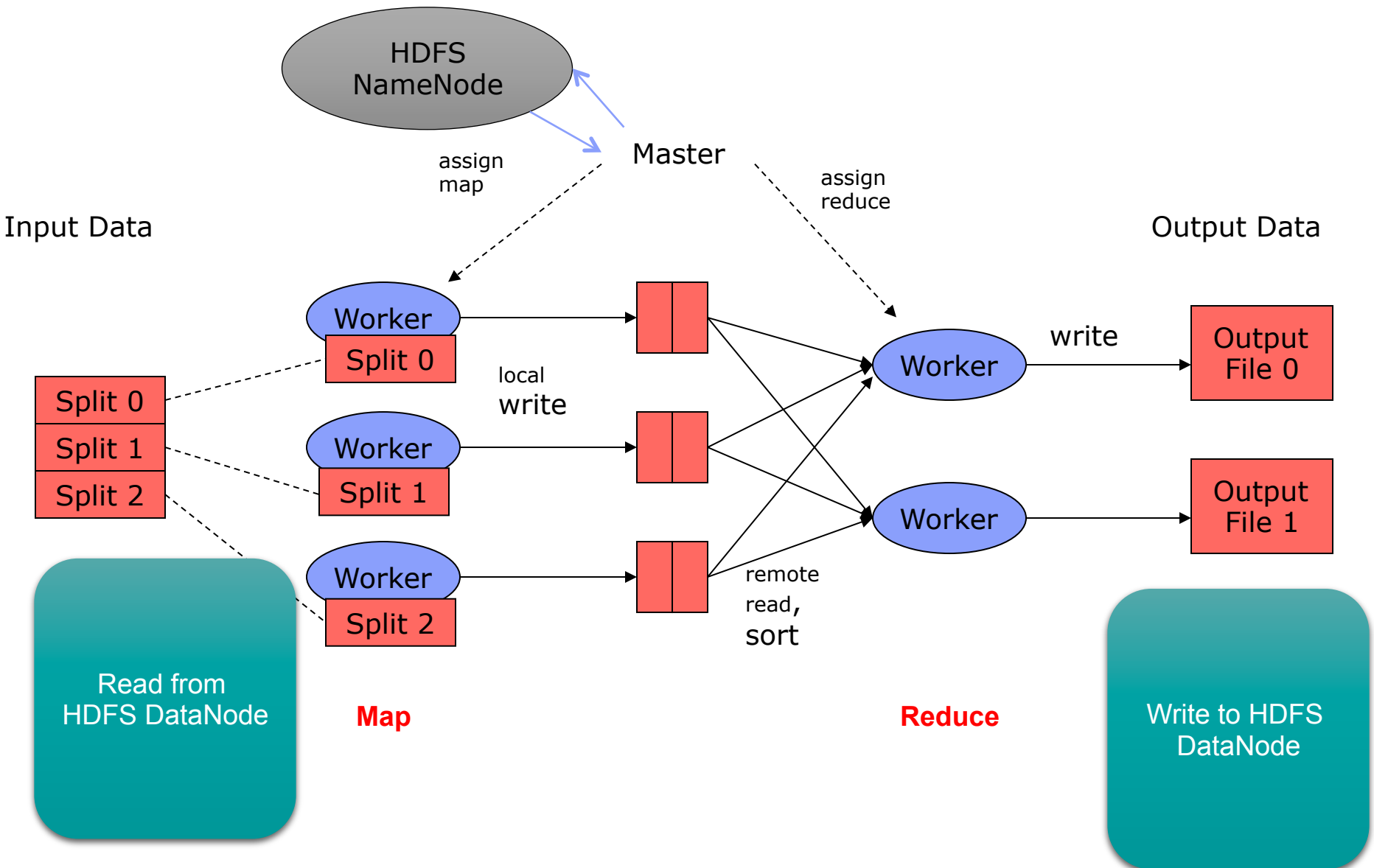
- Accepts the Mapper output, and aggregates values on the key
 - For our example, the reducer input would be:
<The, 1> <teacher, 1> <went, 1> <to, 1> <the, 1> <**store**, 1> <the, 1> <store, 1>
<was, 1> <closed, 1> <the, 1> <**store**, 1> <opens, 1> <in, 1> <the, 1> <morning, 1>
<the 1> <**store**, 1> <opens, 1> <at, 1> <9am, 1>
 - The output would be:
<The, 6> <teacher, 1> <went, 1> <to, 1> <**store**, 3> <was, 1> <closed, 1> <opens,
1> <morning, 1> <at, 1> <9am, 1>

Another example: Chaining MapReduce

Count of URL access frequency

- Input: Log of accessed URLs, e.g., from proxy server
- Output: For each URL, % of total accesses for that URL
- First step:
 - Map – process web log and outputs $\langle \text{URL}, 1 \rangle$
 - Multiple Reducers - Emits $\langle \text{URL}, \text{URL_count} \rangle$
(So far, like Wordcount. But still need %)
- Chain another MapReduce job after above one
 - Map – processes $\langle \text{URL}, \text{URL_count} \rangle$ and outputs $\langle 1, (\langle \text{URL}, \text{URL_count} \rangle) \rangle$
 - 1 Reducer – Does two passes. In first pass, sums up all URL_count 's to calculate overall_count . In second pass calculates %'s
Emits multiple $\langle \text{URL}, \text{URL_count}/\text{overall_count} \rangle$

MapReduce is tightly integrated with HDFS (Hadoop File System)



Data locality is important for performance

- **Master scheduling policy:**

- Asks HDFS for locations of replicas of input file blocks
- Map tasks scheduled so HDFS input block replica are on same machine or same rack

- **Effect:** Thousands of machines read input at local disk speed

- Don't need to transfer input data all over the cluster over the network: eliminate network bottleneck!

Failure in MapReduce

- Failures are the norm in data centers
- **Worker failure**
 - Master detects if workers failed by periodically pinging them (this is called a “heartbeat”)
 - Re-execute in-progress map/reduce tasks
- **Master failure**
 - Single point of failure; Resume from Execution Log
- **Robust**
 - Google’s experience: lost 1600 of 1800 machines once, but finished fine

Refinement: Redundant Execution

- Slow workers or **stragglers** significantly lengthen completion time
 - Slowest worker can determine the total latency!
 - This is why many systems measure **99th percentile latency**
 - Other jobs consuming resources on machine
 - Bad disks with errors transfer data very slowly
- **Solution:** spawn backup copies of tasks
 - Whichever one finishes first "wins"
 - I.e., treat slow executions as failed execute

Borrowed from Indranil Gupta, Faria Kalim, Patrick Wendell

Spark



Motivation

- Map-reduce based tasks are slow
 - Data written to and read from storage
 - In the beginning and end of each Map and Reduce task
- Iterative algorithms not supported
 - Need to chain map reduce jobs → cumbersome, need to know how many jobs in advance (hard to do a loop)
- No support for interactive queries (can take hours or days to complete)

Spark's Key Concept: Resilient Distributed Datasets (RDDs)

- A form of **distributed shared memory**
 - Eliminates the need to read/write to/from disk intermediate data between iterations
 - Read only / immutable, partitioned collections of records in memory
 - Deterministic
 - Formed by specific operations (map, filter, join, etc.)
 - Can be read from stable storage or other RDDs
- More expressive interface than MapReduce
 - Transformations (e.g. map, filter, groupBy)
 - Actions (e.g. count, collect, save)
- Recent versions of Spark introduced Datasets/Dataframes
 - Like an RDD, but you can run SQL-like queries over it
 - Organized into rows columns, similar to relations in databases

Spark programming interface

- Lazy operations
 - Transformations not done until action
- Operations on RDDs
 - Transformations - build new RDDs
 - Can include both traditional map and/or reduce operations
 - Actions - compute and output results
 - E.g., to a file, to a Python collection
- Partitioning – layout across nodes
- Persistence – final output can be stored on disk

RDD on Spark

Transformations	$map(f : T \Rightarrow U)$: $RDD[T] \Rightarrow RDD[U]$ $filter(f : T \Rightarrow Bool)$: $RDD[T] \Rightarrow RDD[T]$ $flatMap(f : T \Rightarrow Seq[U])$: $RDD[T] \Rightarrow RDD[U]$ $sample(fraction : Float)$: $RDD[T] \Rightarrow RDD[T]$ (Deterministic sampling) $groupByKey()$: $RDD[(K, V)] \Rightarrow RDD[(K, Seq[V])]$ $reduceByKey(f : (V, V) \Rightarrow V)$: $RDD[(K, V)] \Rightarrow RDD[(K, V)]$ $union()$: $(RDD[T], RDD[T]) \Rightarrow RDD[T]$ $join()$: $(RDD[(K, V)], RDD[(K, W)]) \Rightarrow RDD[(K, (V, W))]$ $cogroup()$: $(RDD[(K, V)], RDD[(K, W)]) \Rightarrow RDD[(K, (Seq[V], Seq[W]))]$ $crossProduct()$: $(RDD[T], RDD[U]) \Rightarrow RDD[(T, U)]$ $mapValues(f : V \Rightarrow W)$: $RDD[(K, V)] \Rightarrow RDD[(K, W)]$ (Preserves partitioning) $sort(c : Comparator[K])$: $RDD[(K, V)] \Rightarrow RDD[(K, V)]$ $partitionBy(p : Partitioner[K])$: $RDD[(K, V)] \Rightarrow RDD[(K, V)]$
Actions	$count()$: $RDD[T] \Rightarrow Long$ $collect()$: $RDD[T] \Rightarrow Seq[T]$ $reduce(f : (T, T) \Rightarrow T)$: $RDD[T] \Rightarrow T$ $lookup(k : K)$: $RDD[(K, V)] \Rightarrow Seq[V]$ (On hash/range partitioned RDDs) $save(path : String)$: Outputs RDD to a storage system, <i>e.g.</i> , HDFS

Table 2: Transformations and actions available on RDDs in Spark. Seq[T] denotes a sequence of elements of type T.

Example: Log Mining

Load error messages from a log into memory, then interactively search for various patterns

Transformed RDD

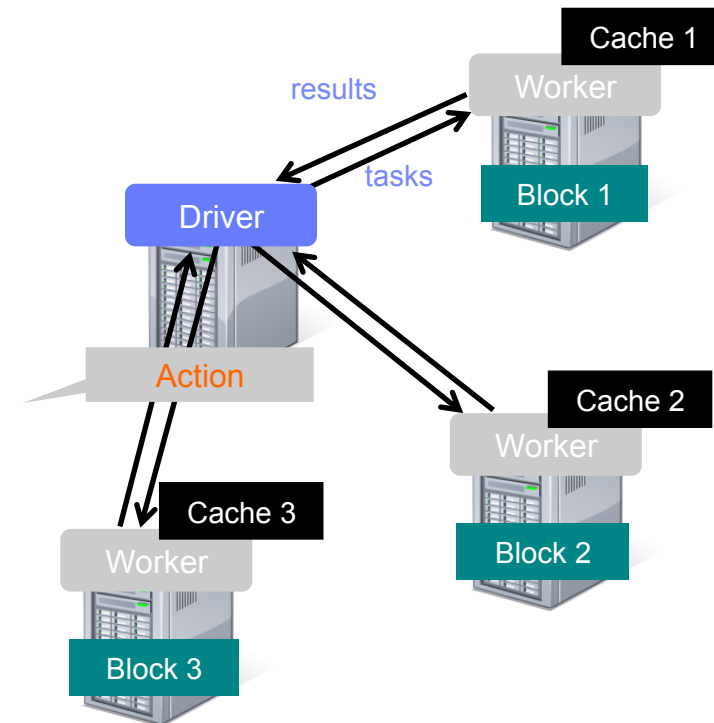
```
lines = spark.textFile("hdfs://...")
errors = lines.filter(lambda s: s.startswith("ERROR"))
messages = errors.map(lambda s: s.split("\t")[2])
messages.cache()
```

```
messages.filter(lambda s: "mysql" in s).count()
messages.filter(lambda s: "php" in s).count()
```

...

Full-text search of Wikipedia

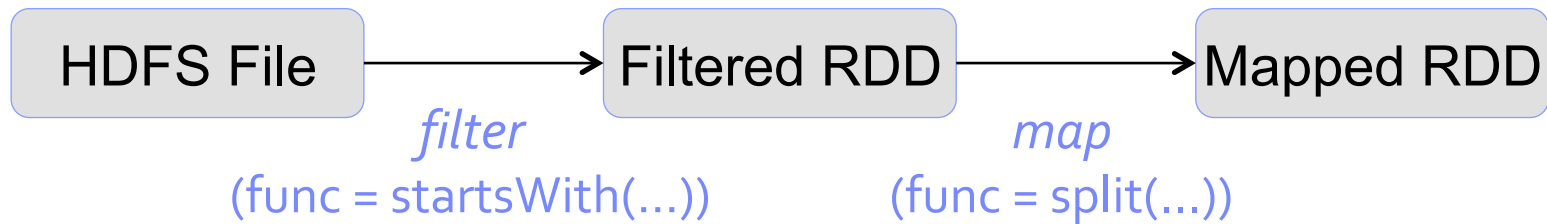
- 60GB on 20 EC2 machine
- 0.5 sec vs. 20s for on-disk



Fault Recovery

RDDs track **lineage** information that can be used to efficiently recompute lost data

```
msgs = textFile.filter(lambda s: s.startsWith("ERROR"))  
               .map(lambda s: s.split("\t")[2])
```



Creating RDDs

Parallelize turns a Python collection into an RDD

```
> sc.parallelize([1, 2, 3])
```

Load text file from local FS, HDFS, or S3

```
> sc.textFile("file.txt")
```

```
> sc.textFile("directory/*.txt")
```

```
> sc.textFile("hdfs://namenode:9000/path/file")
```

Basic Transformations

```
> nums = sc.parallelize([1, 2, 3])
```

```
# Pass each element through a function
```

```
> squares = nums.map(lambda x: x*x) // {1, 4, 9}
```

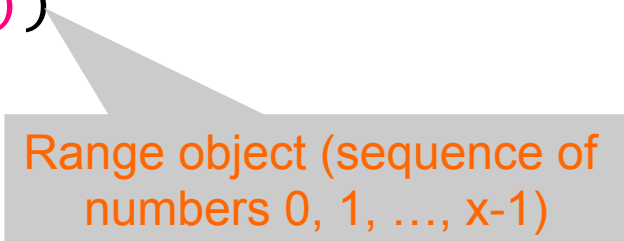
```
# Keep elements passing a predicate
```

```
> even = squares.filter(lambda x: x % 2 == 0) // {4}
```

```
# Map each element to zero or more others
```

```
> nums.flatMap(lambda x: => range(x))
```

```
> # => {0, 0, 1, 0, 1, 2}
```



Range object (sequence of numbers 0, 1, ..., x-1)

Basic Actions

```
> nums = sc.parallelize([1, 2, 3])  
# Retrieve RDD contents as a local collection  
> nums.collect() # => [1, 2, 3]  
# Return first K elements  
> nums.take(2)    # => [1, 2]  
# Count number of elements  
> nums.count()    # => 3  
# Merge elements with an associative function  
> nums.reduce(lambda x, y: x + y) # => 6  
# Write elements to a text file  
> nums.saveAsTextFile("hdfs://file.txt")
```

Working with Key-Value Pairs

Spark's “distributed reduce” transformations operate on RDDs of key-value pairs

Python:

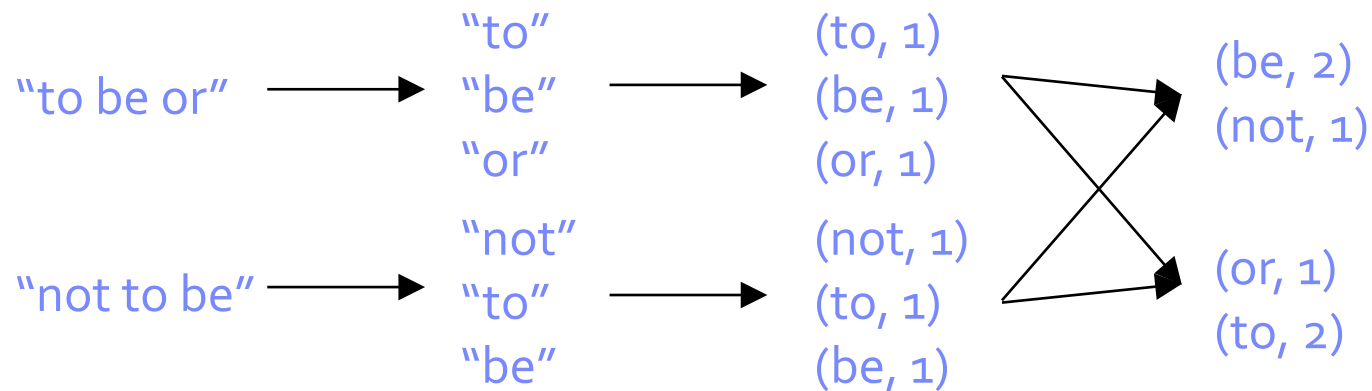
```
pair = (a, b)
pair[0] # => a
pair[1] # => b
```

Some Key-Value Operations

```
> pets = sc.parallelize(  
    [("cat", 1), ("dog", 1), ("cat", 2)])  
  
> pets.reduceByKey(lambda x, y: x + y)  
    # => {(cat, 3), (dog, 1)}  
  
> pets.groupByKey() # => {(cat, [1, 2]), (dog, [1])}  
  
> pets.sortByKey()  # => {(cat, 1), (cat, 2), (dog, 1)}
```

Example: Word Count

```
> lines = sc.textFile("hamlet.txt")  
> counts = lines.flatMap(lambda line: line.split(" "))  
                    .map(lambda word => (word, 1))  
                    .reduceByKey(lambda x, y: x + y)
```



More RDD Operators

- map
- filter
- groupBy
- sort
- union
- join
- leftOuterJoin
- rightOuterJoin

- reduce
- count
- fold
- reduceByKey
- groupByKey
- cogroup
- cross
- zip

sample
take
first
partitionBy
mapWith
pipe
save ...

SQL on Spark

- Spark SQL allows you to use SQL on Spark
- Instead of using RDDs, it uses DataFrames
 - Like an RDD, but in a table format
 - Each column has a name
- A few useful operations:
- `df.collect()`
 - `[Row(price=100, company='Ford'), Row(price=5, company='VW')]`
- `df.columns`
 - `['price', 'company']`
- `df.count()`
 - `2`
- `df.filter(df.price > 50).collect()`
 - `[Row(price=100, company='Ford')]`

Return records as a list

Returns column format

Returns number of rows

Filters

User-Defined Functions (UDF) in Spark

- Sometimes you need to write a custom operation that is run on each row
 - Cleaning/mapping/filtering strings
 - Custom math operation
- For example:
 - For a column that contains names, remove all middle names and just keep first and last name
 - Extract the name of the service from an error log
 - Change the names “Nick, Rick” to “Nicholas, Richard”
- UDF allows you to define a new custom function that will be applied to each row
 - Word of caution: be sure to consider NULL case
 - Most commonly within the UDF itself
 - Need to register UDF function (different name than the Python function)

UDF Example

```
from pyspark.sql.types import StringType
from pyspark.sql.functions import udf, col

def clean_middle(name):
    split_name = name.split()
    if (len(split_name) > 2):
        return split_name[0] + " " + split_name[-1]
    else:
        return name

clean_middle_udf = udf(lambda name: clean_middle(name), StringType())
names = [{"name": "Jane Smith"}, {"name": "John J Smith"}]
df = spark.createDataFrame(names)
df.withColumn("no_middle", clean_middle_udf(col("name"))).show()
```

```
+-----+-----+
|      name| no_middle|
+-----+-----+
| Jane Smith| Jane Smith|
| John J Smith| John Smith|
+-----+-----+
```

Under The Hood: DAG Scheduler

- General task graphs
- Automatically pipelines functions
- Data locality aware
- Partitioning aware to avoid shuffles

